

CPE333 Operating Systems (1/2026)

Problem Session 2: Process Creation & Pipes

1. Name: thnabun	ID: _____
2. Name: _____	ID: _____
3. Name: _____	ID: _____
4. Name: _____	ID: _____

Environment: Ubuntu 24.04 on WSL2, gcc 15.2.0, 8 CPUs

1 fork() examples (P1)

Two programs were written following the `fork()` example from the lecture slide. Program 1.1 (`p1a.c`) is the plain `fork()` example. Program 1.2 (`p1b.c`) is the same program modified with the `wait()` function in the parent.

1.1 helloworld (p1a.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(int argc, char *argv[])
6 {
7     printf("hello world (pid:%d)\n", (int) getpid());
8     int rc = fork();
9     if (rc < 0) {
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) {
13         printf("hello, I am child (pid:%d)\n", (int) getpid());
14     } else {
15         printf("hello, I am parent of %d (pid:%d)\n", rc, (int) getpid());
16     }
17     return 0;
18 }
```

How it works: `fork()` creates a child process that is an almost exact copy of the parent. In the child the return value `rc` is 0; in the parent `rc` is the child's PID. Each branch prints its own message; both processes continue running concurrently after `fork()`.

1.2 modified with wait() (p1b.c)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(int argc, char *argv[])
7 {
8     printf("hello world (pid:%d)\n", (int) getpid());
9     int rc = fork();
10    if (rc < 0) {
11        fprintf(stderr, "fork failed\n");
```

```

12     exit(1);
13 } else if (rc == 0) {
14     printf("hello, I am child (pid:%d)\n", (int) getpid());
15 } else {
16     int wc = wait(NULL);
17     printf("hello, I am parent of %d (wc:%d) (pid:%d)\n", rc, wc, (int)
getpid());
18 }
19 return 0;
20 }

```

How it works: the parent calls `wait(NULL)`, which blocks (sleeps) until the child terminates and then reaps the child's exit status. Only after that does the parent print its message and exit.

Results (three runs of each)

```

=== P1a: fork without wait ===
--- run 1 ---
hello world (pid:1237)
hello, I am parent of 1238 (pid:1237)
hello world (pid:1237)
hello, I am child (pid:1238)
--- run 2 ---
hello world (pid:1239)
hello, I am parent of 1240 (pid:1239)
hello world (pid:1239)
hello, I am child (pid:1240)
--- run 3 ---
hello world (pid:1241)
hello, I am parent of 1242 (pid:1241)
hello world (pid:1241)
hello, I am child (pid:1242)

=== P1b: fork with wait ===
--- run 1 ---
hello world (pid:1243)
hello, I am child (pid:1244)
hello world (pid:1243)
hello, I am parent of 1244 (wc:1244) (pid:1243)
--- run 2 ---
hello world (pid:1245)
hello, I am child (pid:1246)
hello world (pid:1245)
hello, I am parent of 1246 (wc:1246) (pid:1245)

```

Discussion

- **Difference 1 – ordering:** In `p1a` the order of the parent's and child's messages is decided by the CPU scheduler, so it can change between runs (here the parent always won the race, but it is non-deterministic in general). In `p1b` the parent blocks inside `wait()` until the child has finished, so the child's message always appears before the parent's message. `wait()` makes the output deterministic and also serialises the two processes.
- **Difference 2 – duplicated line:** In both programs the line "hello world (pid:...)" is printed twice, even though `printf()` for it is executed only once, before `fork()`. **Reason:** `printf()` only writes into the stdio user-space buffer; the buffer is flushed when it is full or at `exit()`. When stdout is a pipe (as when output is captured), it is fully buffered, so at the moment of `fork()` the buffer still holds the text. The child inherits a copy of the parent's buffer, and both processes flush their own copy when they exit, so the line appears twice.

Calling `fflush(stdout)` before `fork()` would avoid this. This is an important subtlety of `fork()`: everything in memory is copied, including hidden library buffers.

- **Difference 3 – cleanup:** In **p1a** the parent does not wait for the child, so if the child died first it would stay as a zombie until the parent exits (see Section 2). In **p1b** `wait()` reaps the child immediately (`wc = 1244` shows `wait()` returned the child's PID).

2 sleep() + ps: zombie and orphan scenarios (P2)

2.1 Child dies while parent is still running (p2a.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 /* Scenario 2.1: child dies first, parent keeps running (sleeps).
6    Child becomes a zombie (<defunct>) until parent exits/reaps. */
7 int main(int argc, char *argv[])
8 {
9     int rc = fork();
10    if (rc < 0) {
11        fprintf(stderr, "fork failed\n");
12        exit(1);
13    } else if (rc == 0) {
14        printf("child (pid:%d) exiting now\n", (int) getpid());
15        exit(0);
16    } else {
17        printf("parent (pid:%d) of child %d sleeping 20s...\n", (int) getpid(),
18              rc);
19        printf("while parent sleeps, run: ps -o pid,ppid,stat,cmd -ef | grep
20              p2a\n");
21        sleep(20);
22        printf("parent (pid:%d) done sleeping, exiting (child was reaped)\n", (
23              int) getpid());
24    }
25    return 0;
26 }

```

2.2 Parent dies while child is still running (p2b.c)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 /* Scenario 2.2: parent dies first, child keeps running (sleeps).
6    Child becomes an orphan, reparented to init/systemd. */
7 int main(int argc, char *argv[])
8 {
9     int rc = fork();
10    if (rc < 0) {
11        fprintf(stderr, "fork failed\n");
12        exit(1);
13    } else if (rc == 0) {
14        printf("child (pid:%d) sleeping 20s, parent will die first...\n", (int)
15              getpid());
16        printf("while child sleeps, run: ps -o pid,ppid,stat,cmd -ef | grep p2b
17              \n");
18        sleep(20);
19        printf("child (pid:%d) done sleeping, exiting\n", (int) getpid());
20    } else {
21        printf("parent (pid:%d) of child %d exiting now\n", (int) getpid(), rc)
22        ;
23        exit(0);
24    }
25    return 0;
26 }

```

Results

```

=== P2a: child dies first, parent sleeps 20 s ===
child (pid:1258) exiting now
parent (pid:1256) of child 1258 sleeping 20s...
ps while the parent sleeps:
  PID  PPID STAT COMMAND
  1256  1255 S+   \_ p2a          <- parent still alive (S = sleeping)
  1258  1256 Z+   |   \_ p2a          <- child DEAD but still present (Z = zombie)
after the parent exits, the entry disappears.

=== P2b: parent dies first, child sleeps 20 s ===
parent (pid:1263) of child 1265 exiting now
child (pid:1265) sleeping 20s, parent will die first...
ps while the child sleeps:
  PID  PPID STAT COMMAND
  1265  1254 S+   p2b          <- only the CHILD exists; PPID is no
                                longer 1263 but 1254 (the session
                                shell / subreaper). On a standard
                                Linux system the child is
                                reparented to init/systemd (PID 1).
after 20 s the child finishes and exits normally; no <defunct> entry.

```

Discussion

- Scenario 2.1 (Zombie / Defunct):** The child exited (state Z), but its parent is still alive and has not called `wait()`, so the kernel cannot discard the child's `task_struct` – it must keep the exit status until someone reads it. `ps` shows the child as `<defunct>` / state Z. The zombie does not consume CPU or memory pages (its address space was already freed), it only keeps a small PCB/PID entry. A zombie cannot be killed with `SIGKILL` – it is already dead – it can only be reaped with `wait()/waitpid()`. Here the zombie disappears when the parent exits: the kernel reparents it to `init`, and `init` (PID 1) automatically reaps it.
 - **Cause:** Parent finished or continues running without calling `wait()`.
 - **Consequence:** If a long-lived parent keeps forking children and never waits for them, zombies accumulate and may exhaust the PID table.
- Scenario 2.2 (Orphan Process):** The parent exited while the child was still running. The kernel reparents the child to `init/systemd` (in this WSL session the child was adopted by the session shell acting as a subreaper, hence PPID 1254). The child keeps running normally (state S, sleeping in `sleep()`) and exits cleanly after its 20 s; since `init` always reaps, no zombie remains.
- Key difference:** A zombie is dead-but-unreaped and does nothing; an orphan is a fully alive, running process whose parent happened to die first – it is simply adopted and continues executing. Both phenomena are consequences of the parent not waiting (or dying before the child).

3 How many processes can we fork? (P3)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  /* Recursive fork counter.
7   Parent waits for child, child forks a child, ... until fork()
8   fails. Every process counts its own depth (the number of successful
9   fork() calls on its path). The deepest process -- the one whose
10  fork() fails -- writes the total to /tmp/forkcount.txt and reports
11  it. Parents then exit one by one (cascade), and main() reads the
12  file and prints the final total.
13  CAUTION: run under a controlled process limit (e.g. 'ulimit -u')
14  so the chain cannot grow without bound. */
15
16 static int depth = 0;
17
18 void fork_recursive(void)
19 {
20     int rc = fork();
21     if (rc < 0) {
22         /* cannot fork anymore: report and record the total */
23         FILE *f = fopen("/tmp/forkcount.txt", "w");
24         if (f) {
25             fprintf(f, "%d\n", depth);
26             fclose(f);
27         }
28         printf("fork() #%d failed at pid %d: cannot fork anymore\n",
29               depth, (int) getpid());
30         printf("TOTAL successful fork() calls: %d\n", depth);
31         exit(0);
32     } else if (rc == 0) {
33         depth++;           /* this fork() succeeded */
34         fork_recursive();  /* child keeps going deeper */
35         exit(0);          /* never reached */
36     } else {
37         wait(NULL);       /* wait for the whole subtree */
38         exit(0);
39     }
40 }
41
42 int main(void)
43 {
44     printf("fork counter started (pid:%d)\n", (int) getpid());
45     fflush(stdout); /* children inherit the buffer: flush first */
46     fork_recursive();
47
48     int total = -1;
49     FILE *f = fopen("/tmp/forkcount.txt", "r");
50     if (f) {
51         fscanf(f, "%d", &total);
52         fclose(f);
53     }
54     printf("FINAL total successful fork() calls: %d\n", total);
55     return 0;
56 }

```

How it works: `fork_recursive()` forks a child; the parent waits, while the child increments its own depth counter and recurses, so only one new process is created per level (a chain, not a bomb). The deepest process is the one whose `fork()` fails; it writes the total number of

successful `fork()` calls to `/tmp/forkcount.txt` and reports it. The parents then exit one by one and `main()` reads the file and prints the final total. Because the chain can grow to thousands of processes, the experiment was run under a controlled process limit (`ulimit -u 1000`) so that `fork()` fails quickly with `EAGAIN` and the program terminates by itself (the assignment warns that an uncontrolled fork loop can crash the system).

Results (per-user process limit `ulimit -u = 1000`)

```
=== P3a: fork until failure, no extra load ===
baseline processes: 33
fork counter started (pid:15990)
fork() #963 failed at pid 16953: cannot fork anymore
TOTAL successful fork() calls: 963

=== P3b: with 10 background 'stay-running' programs (sleep 300) ===
background jobs: 10, processes: 40
TOTAL successful fork() calls: 953 (-10 vs P3a)

=== P3c: after stopping the background programs ===
processes after kill: 30
TOTAL successful fork() calls: 963 (back to P3a level)
```

System limits observed

```
per-user max processes (ulimit -u) : 30283
kernel threads-max                : 60567
kernel pid_max                     : 4194304
```

Discussion

The number of processes a program can fork is bounded by (a) the per-user process limit (`ulimit -u = 30283` here – when it is reached, `fork()` returns `-1` with `errno = EAGAIN`), and (b) available memory.

With the 1000-process cap, the program managed 963 successful forks; the remaining ~ 37 slots were already occupied by the shell, the counter program itself and system daemons (baseline 33). In P3b, the 10 extra background processes took exactly 10 slots, so the counter dropped to 953. After stopping the background programs their slots were freed and the counter returned to 963: fork count tracks the number of free process slots, 1:1. So the answer is: as many as the OS allows – about 30283 minus whatever else runs, or less if memory is the binding constraint (each process needs kernel structures and at least a minimal memory image).

Safety note: The assignment's warning is real. An uncontrolled fork loop keeps creating processes; when the limit is reached it does not stop on its own – if processes are killed to free slots, the loop simply forks again. During this lab a mid-run kill turned the chain into a self-perpetuating fork loop that had to be stopped with repeated `SIGKILL` sweeps, demonstrating how easily an unguarded fork program can exhaust the system. The lesson: always bound fork loops with a limit or a max-depth check (here: `ulimit -u`).

4 Pipe between parent and child (P4)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <sys/wait.h>
6
7 #define BUF_SIZE 128
8
9 int main(int argc, char *argv[])
10 {
11     int fd[2];
12     if (pipe(fd) < 0) {
13         fprintf(stderr, "pipe failed\n");
14         exit(1);
15     }
16     int rc = fork();
17     if (rc < 0) {
18         fprintf(stderr, "fork failed\n");
19         exit(1);
20     } else if (rc == 0) {
21         /* child = sender */
22         close(fd[0]); /* close read-end */
23         char msg[BUF_SIZE];
24         snprintf(msg, sizeof(msg), "Hello from child PID: %d", (int) getpid());
25         printf("Child: Child PID: %d\n", (int) getpid());
26         write(fd[1], msg, strlen(msg) + 1);
27         close(fd[1]);
28         exit(0);
29     } else {
30         /* parent = receiver */
31         close(fd[1]); /* close write-end */
32         printf("Parent: Parent PID: %d\n", (int) getpid());
33         char buf[BUF_SIZE];
34         ssize_t n = read(fd[0], buf, sizeof(buf) - 1);
35         if (n < 0) {
36             fprintf(stderr, "read failed\n");
37             exit(1);
38         }
39         buf[n] = '\0';
40         printf("Parent: %s\n", buf);
41         close(fd[0]);
42         wait(NULL);
43     }
44     return 0;
45 }

```

How it works: `pipe(fd)` creates a unidirectional pipe: `fd[0]` is the read-end, `fd[1]` is the write-end. The file descriptors are inherited by the child through `fork()`, so both processes share the same pipe. To avoid confusion and to let `read()` see EOF correctly, each side closes the end it does not use: the child (sender) closes `fd[0]` and writes its message into `fd[1]`; the parent (receiver) closes `fd[1]` and reads from `fd[0]`. Because all copies of the write-end are closed once the child exits, the parent's `read()` returns the message and then EOF.

Results (matches the expected output)

```

Child: Child PID: 1328
Parent: Parent PID: 1327
Parent: Hello from child PID: 1328

```


5 Pipe edge cases (P5)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  #define BUF_SIZE 128
8
9  /* P5 experiments on pipe behaviour.
10     mode 1: sender tries to read back its own message
11     mode 2: receiver reads before sender sends (with sleeps to force order)
12     mode 3: sender sends several messages before receiver reads */
13 int main(int argc, char *argv[])
14 {
15     if (argc < 2) {
16         fprintf(stderr, "usage: %s <1|2|3>\n", argv[0]);
17         exit(1);
18     }
19     int mode = atoi(argv[1]);
20     int fd[2];
21     if (pipe(fd) < 0) {
22         fprintf(stderr, "pipe failed\n");
23         exit(1);
24     }
25     int rc = fork();
26     if (rc < 0) {
27         fprintf(stderr, "fork failed\n");
28         exit(1);
29     } else if (rc == 0) {
30         /* child = sender */
31         close(fd[0]);
32         char msg[BUF_SIZE];
33         snprintf(msg, sizeof(msg), "Hello from child PID: %d", (int) getpid());
34         printf("Child: Child PID: %d\n", (int) getpid());
35
36         if (mode == 1) {
37             /* 5.1 sender tries to read back its own message from its
38                read-end (which is closed). */
39             char buf[BUF_SIZE];
40             ssize_t n = read(fd[0], buf, sizeof(buf));
41             printf("Child read() returned %zd (errno would be set if n<0)\n", n);
42         }
43         write(fd[1], msg, strlen(msg) + 1);
44     } else if (mode == 3) {
45         /* 5.3 sender sends several messages before receiver reads */
46         for (int i = 0; i < 5; i++) {
47             char m[BUF_SIZE];
48             snprintf(m, sizeof(m), "Message %d from child PID: %d",
49                      i + 1, (int) getpid());
50             write(fd[1], m, strlen(m) + 1);
51             printf("Child sent: %s\n", m);
52         }
53     } else {
54         /* mode 2: parent reads first (handled below) */
55         sleep(2); /* guarantee receiver reads before we send */
56         printf("Child (sender) sends after delay...\n");
57         write(fd[1], msg, strlen(msg) + 1);
58     }
59     close(fd[1]);
60     exit(0);
61 } else {

```

```

61      /* parent = receiver */
62      close(fd[1]);
63      printf("Parent: Parent PID: %d\n", (int) getpid());
64      if (mode == 2) {
65          /* 5.2 receiver reads before sender sends: read() should
66             block until the sender writes (or EOF if sender closes). */
67          printf("Parent: reading (will block until sender writes)...\n");
68      }
69      char buf[BUF_SIZE * 8];
70      ssize_t n;
71      while ((n = read(fd[0], buf, sizeof(buf) - 1)) > 0) {
72          buf[n] = '\0';
73          /* a single read() may return several NUL-separated messages */
74          char *p = buf;
75          while (p < buf + n) {
76              printf("Parent received: %s\n", p);
77              p += strlen(p) + 1;
78          }
79      }
80      printf("Parent: read() returned %zd (EOF -> sender closed pipe)\n", n);
81      close(fd[0]);
82      wait(NULL);
83  }
84  return 0;
85  }

```

5.1 Sender tries to read back its own message (5.1)

```

Child: Child PID: 1330
Child read() returned -1 (errno would be set if n<0)
Parent: Parent PID: 1329
Parent received: Hello from child PID: 1330
Parent: read() returned 0 (EOF -> sender closed pipe)

```

Observed: The child's `read(fd[0], ...)` immediately returned `-1`. Since the child had closed its read-end (`fd[0]`) before communicating, the descriptor is invalid for the child, and `read()` fails with `errno = EBADF` (bad file descriptor) without blocking. The message itself was then still sent and received normally.

Conclusion: A process can only read from a read-end it still has open; after `close()` the descriptor cannot be used again.

5.2 Receiver reads before sender sends (5.2)

```

Child: Child PID: 1332
Child (sender) sends after delay...
Parent: Parent PID: 1331
Parent: reading (will block until sender writes)...
Parent received: Hello from child PID: 1332
Parent: read() returned 0 (EOF -> sender closed pipe)

```

Observed: The parent called `read()` before the child wrote anything (the child deliberately slept 2 s first). `read()` did not fail or return 0 – it blocked until the child wrote the message, then returned it.

Conclusion: Reading an empty pipe blocks the reader until data arrives (or until every write-end is closed, which makes `read()` return 0 = EOF). No data is lost and no error occurs; the pipe synchronises the two processes (producer/consumer).

5.3 Sender sends several messages before receiver reads (5.3)

```
Child: Child PID: 1334
Child sent: Message 1 from child PID: 1334
Child sent: Message 2 from child PID: 1334
Child sent: Message 3 from child PID: 1334
Child sent: Message 4 from child PID: 1334
Child sent: Message 5 from child PID: 1334
Parent: Parent PID: 1333
Parent received: Message 1 from child PID: 1334
Parent received: Message 2 from child PID: 1334
Parent received: Message 3 from child PID: 1334
Parent received: Message 4 from child PID: 1334
Parent received: Message 5 from child PID: 1334
Parent: read() returned 0 (EOF -> sender closed pipe)
```

Observed: The child wrote all five messages before the parent read anything; they were all received afterwards, in the exact order they were sent (FIFO), with none lost or reordered. The kernel buffers the bytes in the pipe's internal buffer (64 KiB by default), so the sender does not block as long as the buffer is not full. Because a pipe is a byte stream (not a message stream), the receiver may get several messages in one `read()` call (all five arrived in the first read here, which is why the loop printed all of them from one buffer). After the sender closed its write-end, `read()` returned 0 (EOF).

Summary

`fork()` copies the process, so children race with the parent unless `wait()` serialises them (P1); every resource, including stdio buffers and pipe descriptors, is shared/copied. A child that dies without being waited for becomes a zombie, and a parent that dies first creates an orphan that `init` adopts (P2). The number of processes one can create is bounded by the per-user limit and memory; other running programs reduce it one-for-one (P3). Pipes allow a parent and child to communicate: the unused end must be closed on each side, read blocks on an empty pipe, data is buffered and delivered in order, and closing the last write-end signals EOF (P4, P5). The most dangerous pitfall is an unguarded fork loop, which can exhaust the system – fork count experiments must be run under a controlled process limit.